# MVPCompanion

## The 15-Stage MVP Process

From App Idea to a Focused, Testable and Buildable MVP

A practical method for progressively transforming an uncertain app idea into a coherent MVP definition.

**Clarify before designing. Model before listing Features. Scope before specifying. Define learning before building.**

# The MVPCompanion 15-Stage Process

Turning an app idea into a real product is rarely difficult because people lack ideas. The difficult part is deciding which idea is worth pursuing, for whom, why it matters, what the product actually needs to do, and what should deliberately be left out.

MVPCompanion takes the opposite approach. Instead of starting with features, it starts with clarity. The process progressively transforms an uncertain idea into a coherent product definition.

**Idea → Problem → User → Value → Product → Lifecycle → Jobs → Features → Experience → Information Architecture → Data → Requirements → Learning → Technology → MVP Boundary**

The result is not simply a smaller feature list. It is a product that has become sufficiently clear that another person can understand what needs to be built, why it needs to be built, and how the first version will be used to learn.

## Why MVPCompanion Uses 15 Stages

The number 15 is not intended to be magical. The purpose of the 15 stages is to create a practical sequence of decision gates. Each stage removes a different type of uncertainty and produces an output that becomes an input for the following stages.

Too few stages hide important decisions inside broad categories. Too many stages fragment the work into bureaucracy. Fifteen provides a useful middle ground: enough structure to prevent important questions from being skipped, without turning the process itself into the work.

| Stage | Core question |
|---|---|
| 1. Idea | What could this be? |
| 2. Problem | What real problem exists? |
| 3. User | Who has the problem? |
| 4. Value | Why does solving it matter? |
| 5. Product | What is the product fundamentally? |
| 6. Lifecycle | How does the product's central object evolve? |
| 7. Jobs | What must the user accomplish? |
| 8. Feature Set | What capabilities are actually required? |
| 9. Experience | How does the user get value? |
| 10. Information Architecture | How should the product be organized? |
| 11. Data Model | What information must the product manage? |
| 12. Requirements | What exactly must the system do? |
| 13. Learning | How will we know whether it works? |
| 14. Technical Boundaries | What technology is required — and what is not? |
| 15. MVP Boundary | What are we actually building now? |

The stages are ordered deliberately. The process moves from strategic uncertainty toward implementation certainty.

# The Principle Behind the Process

The central principle is progressive clarification: start with an uncertain idea, make the problem and user explicit, establish the product model, derive the smallest coherent solution, and finally specify it precisely enough to build and learn from.

A useful traceability chain runs through the entire process:

**Problem → User → Outcome → Job → Capability → Requirement → Acceptance Criterion → Metric**

Every important element of the MVP should be traceable through this chain. If a feature cannot be connected to a meaningful user job or outcome, it deserves to be challenged. If an important outcome has no capability supporting it, the MVP may be incomplete. If a requirement cannot be tested, it is not specified precisely enough.

# The Five Logical Blocks

**1. Understand the Opportunity — Stages 1–4:** Idea, Problem, User, Value. This asks: Is there a meaningful product opportunity here?

**2. Understand the Product — Stages 5–7:** Product, Lifecycle, Jobs. This asks: What is the product, how does it work conceptually, and what must users accomplish?

**3. Design the MVP — Stages 8–12**: Feature Set, Experience, Information Architecture, Data Model, Requirements. This asks: What does the first version actually need to contain and how should it work?

**4. Prepare for Learning and Building — Stages 13–14:** Learning and Technical Boundaries. This asks: How will we learn from the MVP, and what technical foundation is required?

**5. Make the Final Boundary Decision — Stage 15:** MVP Boundary. This asks: What are we actually committing to build?

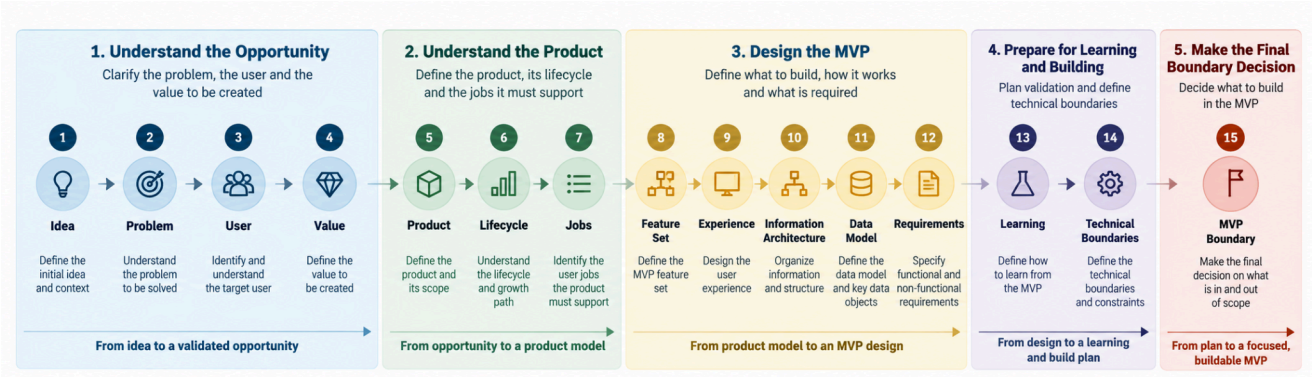



**Figure 1. The MVPCompanion 15-stage process.** Each stage addresses a different category of uncertainty, progressively transforming an initial idea into a focused, testable and buildable MVP definition.

# Stage 1 — Capture the Raw Idea

**Core question: What could this be?**

Every product starts with uncertainty. Capture the trigger, context, intended user and initial hypothesis before polishing the solution.

Write the idea in one sentence. Describe the trigger: personal pain, observed behavior, customer request, market gap or technology opportunity. Separate what you know from what you assume. Record the intended user, even if provisional. State the expected benefit in outcome terms. List the biggest assumptions that would have to be true.

Output: Idea Brief.

Skipping this stage often leads to premature solution design. Stage 1 creates a starting point that can subsequently be challenged.

# Stage 2 — Define the Problem

**Core question: What real problem exists?**

Move from solution language to problem language. The problem should exist independently of the proposed app.

Ask who experiences the problem, what they are trying to accomplish, when it occurs, what makes the current approach difficult, which tools and workarounds are used today, why existing alternatives are insufficient, and what happens if the problem remains unsolved.

A strong problem definition survives even if the proposed solution disappears. This protects the project from building a sophisticated solution to a problem that does not matter enough.

Output: Problem Statement + evidence and assumptions.

# Stage 3 — Define the Primary User

**Core question: Who has the problem?**

Avoid designing for “users” as an abstract group. Choose a primary user whose problem is strong enough to justify changing behavior.

Define one primary persona or segment. Describe context, responsibilities, constraints and goals. Document the current workflow and alternatives. Identify secondary users separately. Explicitly define who is not the initial target and explain why this segment is the right beachhead.

The same problem can have very different value depending on who experiences it. Stage 3 turns a generic problem into a specific product opportunity.

Output: Primary User Profile + Target / Non-Target Segment.

# Stage 4 — Establish the Value Proposition

**Core question: Why does solving it matter?**

Connect problem and user to a compelling outcome. The value proposition should be understandable without a feature list.

A useful formulation is: For [user] who [problem], the product helps them [outcome]. Define the main job-to-be-done, the most important benefits, the differentiating promise, the strongest alternatives and why the product should win against them. A USP is not merely a slogan. It should explain why the proposed product could win.

Output: Jobs + Value Proposition + Benefits + USP + Positioning.

# Stage 5 — Define What the Product Fundamentally Is

**Core question: What is the product fundamentally?**

Before deciding which screens to build, define the product's underlying mental model.

Identify the central entity or object, supporting entities and relationships. Separate durable objects from transient tasks or actions. Decide which information needs structured fields and what can remain free-form. Draw a simple entity/context diagram. Think in objects, not screens. The product model tells us what exists; screens are only one way of presenting it.

Output: Product Mental Model + High-Level Entity Map.

# Stage 6 — Define the Lifecycle

**Core question: How does the product's central object evolve?**

If the central object changes over time, make its lifecycle explicit. List meaningful states, define what qualifies an object to enter each state, identify relevant information and actions, and define transitions.

Keep states operationally useful. Do not create states simply because they sound comprehensive.

A lifecycle gives temporal structure to the product model and prevents the MVP from becoming a collection of disconnected screens.

Output: Lifecycle Model.

# Stage 7 — Identify the Core User Jobs

**Core question: What must users accomplish?**

Jobs are the bridge between product strategy and features. A job describes what the user is trying to accomplish, not the UI element used to accomplish it.

List 3–7 core jobs. Rank them by importance to the value proposition. For each job, define trigger, user action and desired outcome. Identify minimum information required and the decision or result produced. Remove jobs that are interesting but not essential to the first product.

A feature-first process asks “What could the app do?” A job-first process asks “What does the user actually need to accomplish?”

Output: Prioritized Jobs-to-be-Done / Core Use Cases.

# Stage 8 — Derive the Scope (Feature Set)

**Core question: What capabilities are actually required?**

Only after the jobs are clear should you derive capabilities. For each core job, list candidate capabilities, group them into coherent feature areas and classify each as Must Have, Should Have, Later or Not in MVP.

Define the smallest useful version of every Must Have. Prefer features that reuse the same underlying data model. Challenge every feature: Which job does it support? What happens if we remove it? Document exclusions explicitly.

The MVP is defined not only by what is included but also by what is deliberately excluded.

Output: MVP Feature Map + Out-of-Scope List.

# Stage 9 — Design the Core Experience

**Core question: How does the user get value?**

An MVP must provide a complete path from entry to value. A collection of individually useful features is not necessarily a usable MVP.

Define onboarding, the first object the user creates or imports, the first-value moment, the happy path, essential empty states, important error and permission states, and progressive disclosure.

The complete-journey test is simple: Can a new user enter, understand what to do, perform the core job and reach the promised outcome?

Output: Core User Flows + Screen Inventory.

# Stage 10 — Build the Information Architecture

**Core question: How should the product be organized?**

Navigation should reflect the product mental model. Avoid designing navigation as a list of internal departments.

Define top-level navigation, object hierarchy, list and detail views, cross-links and appropriate presentation structures such as dashboards, tables, boards or galleries. Ensure every MVP capability has a natural home.

The information architecture translates the conceptual product into a navigable structure.

Output: Information Architecture + Navigation Model.

# Stage 11 — Define the Data Model

**Core question: What information must the product manage?**

The data model converts the product mental model into something implementable.

List entities, required and optional fields, relationships and cardinality, status/lifecycle fields, ownership boundaries, timestamps and activity/audit needs. Distinguish manually entered, imported and computed data. Remove fields that have no clear purpose.

Every field should have a reason to exist: to support a job, decision, result or system behavior.

Output: MVP Entity / Data Model.

# Stage 12 — Turn Capabilities Into Testable Requirements

**Core question: What exactly must the system do?**

A specification becomes useful when another person can implement it without repeatedly asking what you meant.

Describe observable behavior. Define inputs and outputs, validation rules, empty/loading/success/error/disabled states, permissions and integration behavior. Write acceptance criteria for important features and use examples where behavior could otherwise be ambiguous.

Acceptance criteria expose ambiguity. If a feature cannot be described clearly enough to test, the underlying decision may not yet be clear enough.

Output: Functional Requirements + Acceptance Criteria.

# Stage 13 — Define Metrics and Learning Goals

**Core question: How will we know whether it works?**

An MVP is a learning instrument. “The software works” is not a sufficient success criterion.

Define activation, the core value event, a continued-use or retention signal, one or two quality indicators, important assumptions and learning questions. Define what evidence would cause you to change the product. Metrics are not decoration. They should influence product decisions.

Output: Success Metrics + Learning Questions.

# Stage 14 — Define Technical Boundaries

**Core question: What technology is required — and what is not?**

The MVP specification should be technically concrete enough to estimate and build, but should not prematurely solve every future architecture problem.

State platform/client assumptions. Define backend/API, database/storage, authentication and workspace model, external integrations, AI/service dependencies and major technical risks. Separate known decisions from open technical questions.

The purpose is to establish the technical boundaries of the MVP, not to architect every possible future scale scenario.

Output: High-Level Architecture + Technical Constraints.

# Stage 15 — Establish the MVP Boundary

**Core question: What are we actually building now?**

The MVP is not “everything important, but smaller.” It is the smallest coherent product capable of delivering the core promise and testing the most important assumptions.

List Must Have, Should Have, Later and explicitly Not in MVP. Challenge every Must Have. Check feature dependencies. Then walk through the complete core journey after the cuts. If the user can still enter, perform the core job and receive the promised outcome, the MVP may have found its meaningful boundary. If not, it has been cut too far.

Output: MVP Scope Statement + Prioritized Feature List + Explicit Exclusions.

# Why the Stages Are in This Order

The 15 stages are not simply a checklist. They form a dependency chain. Feature before problem can produce something nobody needs. Feature before the user can optimize for a generic audience. UI before jobs can produce screens that look good but do not help users accomplish anything important. Data model before product model can structure information around implementation convenience rather than product logic. Technical architecture before scope can create infrastructure for capabilities that never belong in the MVP.

The sequence creates progressive constraints. Early stages keep possibilities open. Later stages deliberately narrow them. That is what makes the process efficient.

## Why Not Fewer Than 15?

Different uncertainties require different decisions. Who the user is, what the product is, what information it manages, how it is measured and what technology it requires are related questions, but they are not the same decision. Combining them may make a framework shorter without making the thinking simpler.

Product modeling deserves its own step. Jobs deserve separation from features. Learning deserves explicit attention. Technical boundaries deserve separation from product requirements.

## Why Not More Than 15?

It would be possible to create separate stages for competitive analysis, personas, business model, monetization, UX, visual design, accessibility, analytics, security, APIs, database, deployment and testing. All can matter. But not all deserve to become independent gates in an MVP-definition process.

The 15-stage approach groups related decisions where doing so preserves clarity. This keeps the process comprehensive without turning it into a project-management bureaucracy.

## Each Stage Removes a Different Risk

| Stage | Main uncertainty reduced |
|---|---|
| 1 | What exactly is the idea? |
| 2 | Is there a meaningful problem? |
| 3 | Who experiences it? |
| 4 | Why would they care? |
| 5 | What is the product? |
| 6 | How does it evolve? |
| 7 | What must users accomplish? |
| 8 | What capabilities are necessary? |
| 9 | Can users actually reach value? |
| 10 | Can they understand and navigate it? |
| 11 | Can the product manage the necessary information? |
| 12 | Can it be built unambiguously? |
| 13 | Can we learn whether the hypothesis is true? |
| 14 | Can it be built within reasonable technical boundaries? |
| 15 | What should we actually commit to? |

# The MVPCompanion Difference

MVPCompanion connects the relevant parts of product definition into one practical chain. Its focus is narrower than a complete product-management methodology: take an app idea and make it sufficiently clear, coherent and testable that an MVP can be built deliberately.

**It is a bridge between “I have an idea” and “Here is an MVP that we can build and learn from.”**

# The Most Important Mental Shift

Do not ask first: “What features should my app have?” Ask: “What problem are we solving?” Then: “For whom?” Then: “What outcome matters?” Then: “What is this product fundamentally?” Then: “What must the user accomplish?” Only then: “What capabilities are required?” And finally: “What is the smallest coherent version we can build?”

This sequence may sound slower. In practice, it can make development faster because it reduces expensive uncertainty before code is written.

# The Final MVP Readiness Test

• Can you explain the problem without mentioning the solution?

• Do you know exactly who the primary user is?

• Do you know what outcome that user wants?

• Can you explain the value proposition in one sentence?

• Do you understand what users do today?

• Can you explain what the product fundamentally is?

• Do you understand its central entities and relationships?

• Do you understand how those entities evolve?

• Have you identified the most important user jobs?

• Does every MVP feature support one of those jobs?

• Can the user reach value through a complete end-to-end journey?

• Does the information architecture reflect the product model?

• Is the MVP data model defined?

• Are critical requirements testable?

• Do you know which assumptions the MVP is testing?

• Are the technical boundaries understood?

• Are exclusions explicit?

• Could another person understand what needs to be built?

**If the answer to these questions is yes, the project has crossed an important threshold. It is no longer merely an idea. It has become an MVP definition.**

# The Core Lesson

The most reusable lesson behind MVPCompanion is not a particular feature, framework or document template. It is the sequence of thinking.

**Idea → Problem → User → Value → Product → Lifecycle → Jobs → Features → Experience → Information Architecture → Data → Requirements → Learning → Technology → Boundary**

Each stage removes a different type of uncertainty. The MVP becomes smaller not merely because features are deleted. It becomes smaller because uncertainty has been progressively removed.

There are 15 stages because the journey from an uncertain idea to a buildable MVP contains approximately these 15 meaningful categories of decisions. Fewer stages tend to hide important decisions inside overly broad categories. More stages tend to fragment the process into bureaucracy.

**The goal is simple: make the product clear enough to build — and focused enough to learn from.**

A good MVP is not simply a small application. It is a deliberately bounded experiment in product form. It should deliver a meaningful outcome to a specific user while testing the assumptions that matter most.

## Source basis

This resource is based on the MVP specification guide “From App Idea to MVP Specification”, which defines the progressive-clarification principle and the underlying 15-stage process.